

Rapport Projet Lindenmeyer

Samuel Banwell, Adjil Lo, Thura Htet
Peace Koutchade, Nicole Mvembe

April 2026

Table des matières

1	Introduction	2
1.1	Description générale du projet	2
1.2	Problématique et points difficiles	2
1.2.1	Gestion de la mémoire	2
1.2.2	Définir une grammaire	2
1.2.3	Créer un affichage en 3D	2
2	Structuration du projet	3
2.1	Description des besoins	3
2.2	Fonctionnalités implémentées	3
2.3	Attribution des tâches	4
3	Éléments techniques	4
3.1	Structures de données principales	4
3.1.1	La liste des Symboles (SymbolList)	4
3.1.2	L'ensemble des regles (RuleSet)	4
3.1.3	Le repère 3D de la tortue (heading , left, up)	4
3.1.4	La pile de positions (Deque)	5
3.2	Description des algorithmes	5
3.2.1	L'algorithme de réécriture	5
3.2.2	L'algorithme d'interprétation 2D	5
3.2.3	Les matrices de rotation 3D	6
3.2.4	L'algorithme d'interprétation 3D	6
4	Architecture du projet	8
4.1	Modélisation d'un L-Système	8
4.2	Implémentation d'une tortue	8
4.2.1	Tortue2D	12
4.2.2	Tortue3D	12
4.3	Interface graphique	12
4.3.1	Rendu 3D	12
5	Expérimentations et usages	16
5.1	Performances	16
6	Conclusion	16
6.1	Récapulatif des fonctionnalités principales	16
6.2	Propositions d'améliorations	16

1 Introduction

1.1 Description générale du projet

Dans ce projet de deuxième année de licence informatique, nous avons réalisé un programme Java permettant de créer et d'afficher des figures fractales appelées systèmes de Lindenmeyer, ou L-Systèmes.

Un L-Système est une méthode de dessin basée sur une grammaire : on part d'une chaîne de lettres appelée axiome, et on applique des règles qui remplacent chaque lettre par une autre chaîne. En répétant cette opération plusieurs fois, on obtient une chaîne très longue que l'on dessine avec une tortue graphique.

Le projet comporte deux grandes parties : une partie 2D, où la tortue se déplace dans un plan, et une partie 3D, où la tortue se déplace dans l'espace avec trois axes de rotation.

1.2 Problématique et points difficiles

La principale difficulté du projet était la gestion de la mémoire, l'affichage, définir la grammaire du système, bien organiser le code pour que les règles, les symboles et la tortue soient séparés. Cela nous a amenés à utiliser des patrons de conception comme le Flyweight ou l'Observateur, la 3D.

1.2.1 Gestion de la mémoire

Les L-Systèmes étant un système de réécriture récursif, le nombre d'objets à gérer devient très vite conséquent. Il faut donc appliquer une gestion intelligente du nombre d'objets créés pour représenter le système.

1.2.2 Définir une grammaire

Pour dessiner un L-Système,

1.2.3 Créer un affichage en 3D

Pour la partie 3D, la difficulté principale était de représenter correctement l'orientation de la tortue dans l'espace. Contrairement à la 2D où un seul angle suffit, en 3D il faut gérer trois axes indépendants avec des matrices de rotation.

2 Structuration du projet

2.1 Description des besoins

Le projet doit répondre aux besoins suivants : Représenter un L-Système avec un axiome, des règles de réécriture, et une génération courante. Appliquer les règles de façon itérative pour générer les différentes étapes Interpréter la chaîne générée avec une tortue graphique qui trace des segments Afficher le résultat dans une interface graphique interactive En 3D : gérer l'orientation dans l'espace avec trois axes de rotation (yaw, pitch, roll)

2.2 Fonctionnalités implémentées

Partie 2D — interface Swing Saisie libre d'un axiome et de règles dans l'interface Génération du L-Système sur un nombre d'étapes choisi par l'utilisateur Affichage du dessin avec zoom avant/arrière 10 systèmes prédéfinis : flocon de Koch, arbre fractal, courbe du dragon, triangle de Sierpinski, courbe de Hilbert, courbe de Peano, courbe de Lévy, île de Koch quadratique, plante fractale, variante d'arbre. Sélection de la couleur de dessin parmi 11 couleurs. Mode Play : animation qui dessine le L-Système segment par segment. Historique avec un slider pour revenir à une génération précédente.

Partie 3D - Moteur de rendu JavaFX.

Tortue orientée dans l'espace par un repere (heading, left, up). Trois rotations : yaw autour de l'axe up (+/-), pitch autour de l'axe left (&/^), roll autour de l'axe heading (v/). Generation de segments 3D (coordonnées x, y, z pour chaque point de départ et d'arrivée). Rendu des segments sous forme de cylindres orientés dans JavaFX. Sauvegarde et restauration de position avec les symboles [et]

2.3 Attribution des tâches

Membre	Taches réalisées
Samuel	Classes de règles Liste de symboles Affichage 3D
Adjji	Classes de symboles et tortue 2D Implémentation des coordonnées 3D Classe abstraite d'une tortue 3D Définition des constantes 3D
Peace	Développement de la classe Axiom Définition des méthodes principales de la vue des systèmes en 2D Développement de l'interface principale de l'application
Nicole	Développement Moteur et Interface (GUI) Règles stochastiques et contextuelles
Thura	Création des tests Système de sauvegarde des systèmes

3 Éléments techniques

3.1 Structures de données principales

3.1.1 La liste des Symboles (SymbolList)

La génération de L-système est stockée sous forme d'une liste ordonnée de symboles qui est la structure centrale du programme. À chaque étape on parcourt cette liste, on remplace chaque symbole par son successeur selon les règles, et on obtient une nouvelle liste (PHOTO LISTE DES SYMBOLES)

3.1.2 L'ensemble des regles (RuleSet)

Les règles de réécriture sont stockées dans l'ensemble (HashSet), et chaque règle associe une ou plusieurs symboles à une liste de symboles. Pour trouver le remplacement d'un symboles, on parcourt toutes les règles et on retourne la première qui s'applique.

3.1.3 Le repère 3D de la tortue (heading , left, up)

Pour cette partie 3D, l'orientation de la tortue est représentée par trois vecteurs perpendiculaires entre eux, comme un mini-repère qui se déplace

dans l'espace. Ces vecteurs s'appellent Heading (la direction de marche), Left (la gauche de la tortue), Up (le haut de la tortue).

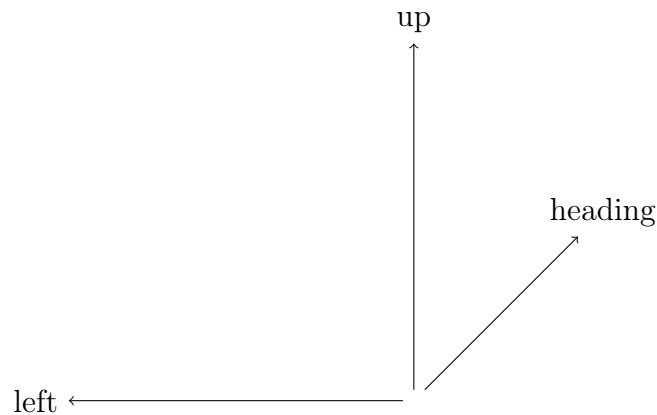


FIGURE 1 – Orientation de la tortue dans un espace 3-dimensionnel

3.1.4 La pile de positions (Deque)

Pour dessiner des branches, on a sauvegarder la position courante, de tracer une branche, puis de revenir au point de départ pour tracer la branche suivante. On utilise pour cela une pile (Deque) qui fonctionne comme une pile d'assiettes et on empile avec [et dépile avec]. En 3D : La pile sauvegarde non seulement la position, mais aussi les trois vecteurs (heading, left, up) pour que la tortue reparte dans la même direction.

3.2 Description des algorithmes

3.2.1 L'algorithme de réécriture

L'algorithme principal du L-Système applique toutes les règles à la génération courante pour produire la suivante. Voici son fonctionnement en pseudo-code :

3.2.2 L'algorithme d'interprétation 2D

Une fois que la chaîne est générée, la tortue la parcourt caractère par caractère. Voici ce que fait chaque symbole

Entrées : une liste de symboles L , un ensemble de règles R

Output : une liste de symboles

début

```
     $tmp \leftarrow$  liste vide;  
    pour  $i = 0$  à longueur de  $L$  faire  
         $symbole \leftarrow L[i]$ ;  
         $left \leftarrow$  symbole à gauche;  
         $right \leftarrow$  symbole à droite;  
         $successeur \leftarrow$   $symbole$ ;  
         $contexte \leftarrow left + symbole + right$ ;  
        pour  $regle \in R$  faire  
            si  $regle$  correspond à  $contexte$  alors  
                 $successeur \leftarrow$  les successeurs spécifiés dans  $regle$ ;  
                sortir de la boucle;  
            fin  
        fin  
    fin  
    retourner  $tmp$ ;
```

fin

Algorithme 1 : Création de la prochaine génération du système

3.2.3 Les matrices de rotation 3D

Pour la 3D, la tortue tourne à l'aide de trois matrices de rotation, une par axe. Ces matrices permettent de modifier les vecteurs heading, left et up de manière cohérente lorsque la tortue change de direction. Les trois rotations sont :

RU(alpha)- yaw : rotation autour de l'axe up, utilisée par + et - , et modifie heading et left.

RL(alpha)- pitch : rotation autour de l'axe left, utilisée par & et ^ , et modifie heading et up.

RH(alpha)- roll : rotation autour de l'axe heading, utilisée par et / , et modifie left et up.

Après chaque rotation, on normalise les vecteurs (on les ramène à une longueur de 1) pour éviter que les erreurs de calcul s'accumulent au fil des itérations.

3.2.4 L'algorithme d'interprétation 3D

l'avancement de la tortue utilise le vecteur heading à la place d'un angle.

Entrées : une liste de symboles, pas et angle des déplacements, un nuancier

Output : une liste de segments correspondants

début

$segments \leftarrow$ liste vide;

$pile \leftarrow$ pile vide;

$x \leftarrow 0$;

$y \leftarrow 0$;

$angle \leftarrow 0$;

pour $symbole \in S$ **faire**

si $symbole \in nuancier$ **alors**

$couleur \leftarrow$ couleur de $symbole$ dans le nuancier;

$nx \leftarrow x + pas * \cos(depAngle)$;

$ny \leftarrow y + pas * \sin(depAngle)$;

$segment \leftarrow \text{Segment}(x, y, nx, ny, couleur)$;

 ajouter $segment$ à $segments$;

$x = nx$;

$y = ny$;

sinon si $symbole = +$ **alors** $angle+ = depAngle$;

sinon si $symbole = -$ **alors** $angle- = depAngle$;

sinon si $symbole = [$ **alors** ajouter $(x, y, angle)$ à la pile;

sinon si $symbole =]$ **alors** restaurer la dernière position de la pile;

fin

retourner $segments$;

fin

Algorithme 2 : Fonctionnement de la tortue 2D

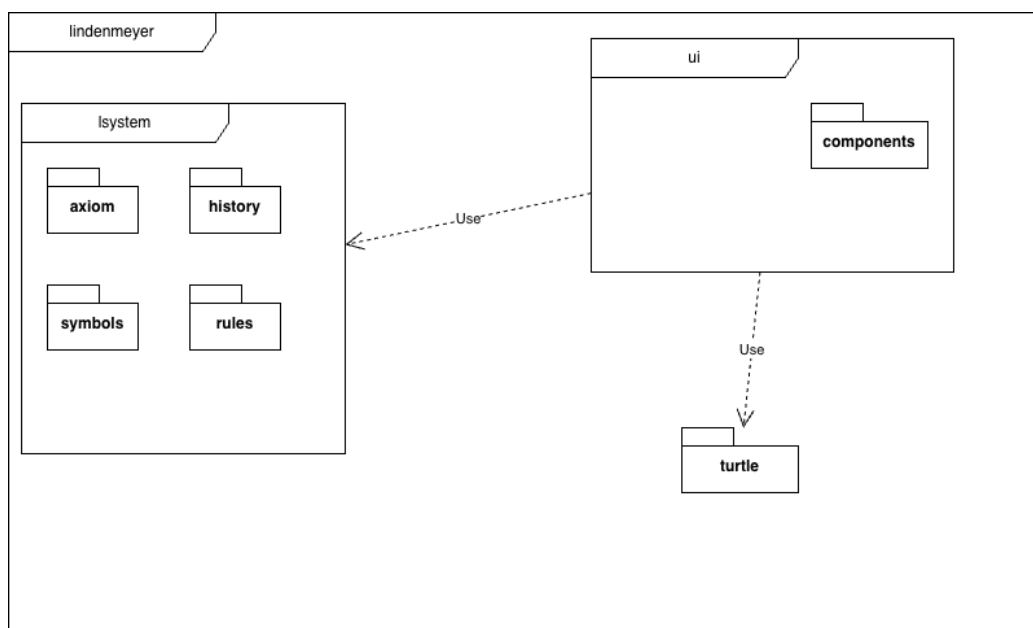


FIGURE 2 – Diagramme du paquetage du projet

4 Architecture du projet

Afin de compléter un projet de cette complexité, une séparation de packages à été fait suivant les responsabilités de chaque package. Ainsi, un package sert à modéliser un système, un autre à fournir les règles de réécriture, et ainsi de suite, tout en restant aussi indépendants que possible.

4.1 Modélisation d'un L-Système

Le modèle du système se base sur quatre packages spécialisés, qui sont ensuite réunis dans l'interface publique de la classe `LSystem` exposée par le package. Cette structure permet de spécialiser chaque composant du L-Système et de les tester indépendamment.

Plusieurs classes usine permettent de créer les règles de réécriture, des symboles, et le L-Système lui-même, facilitant pour les symboles une conception poids-mouche.

4.2 Implémentation d'une tortue

Deux classes, `Tortue` et `Tortue3D`, permettent respectivement sur deux et trois dimensions d'associer à des symboles un ou des mouvements d'une

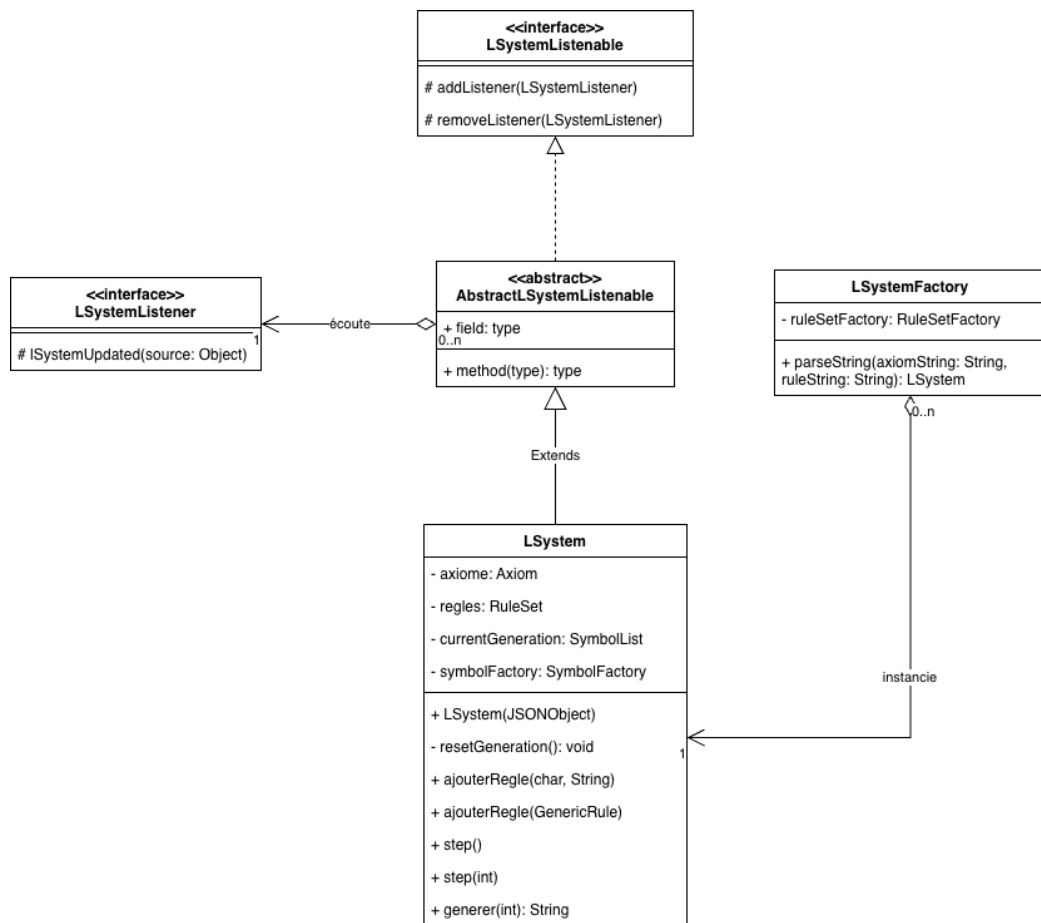


FIGURE 3 – Diagramme des classes du package lsystem

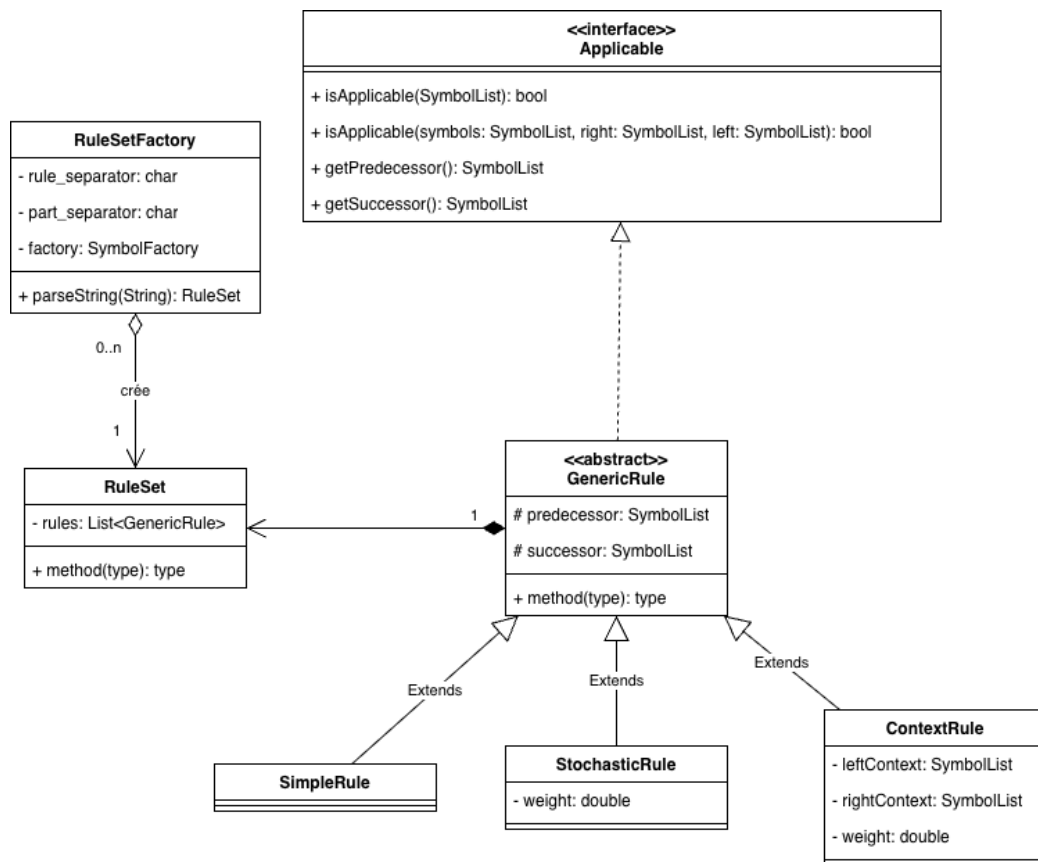


FIGURE 4 – Diagramme des classe du package lsystem.rules

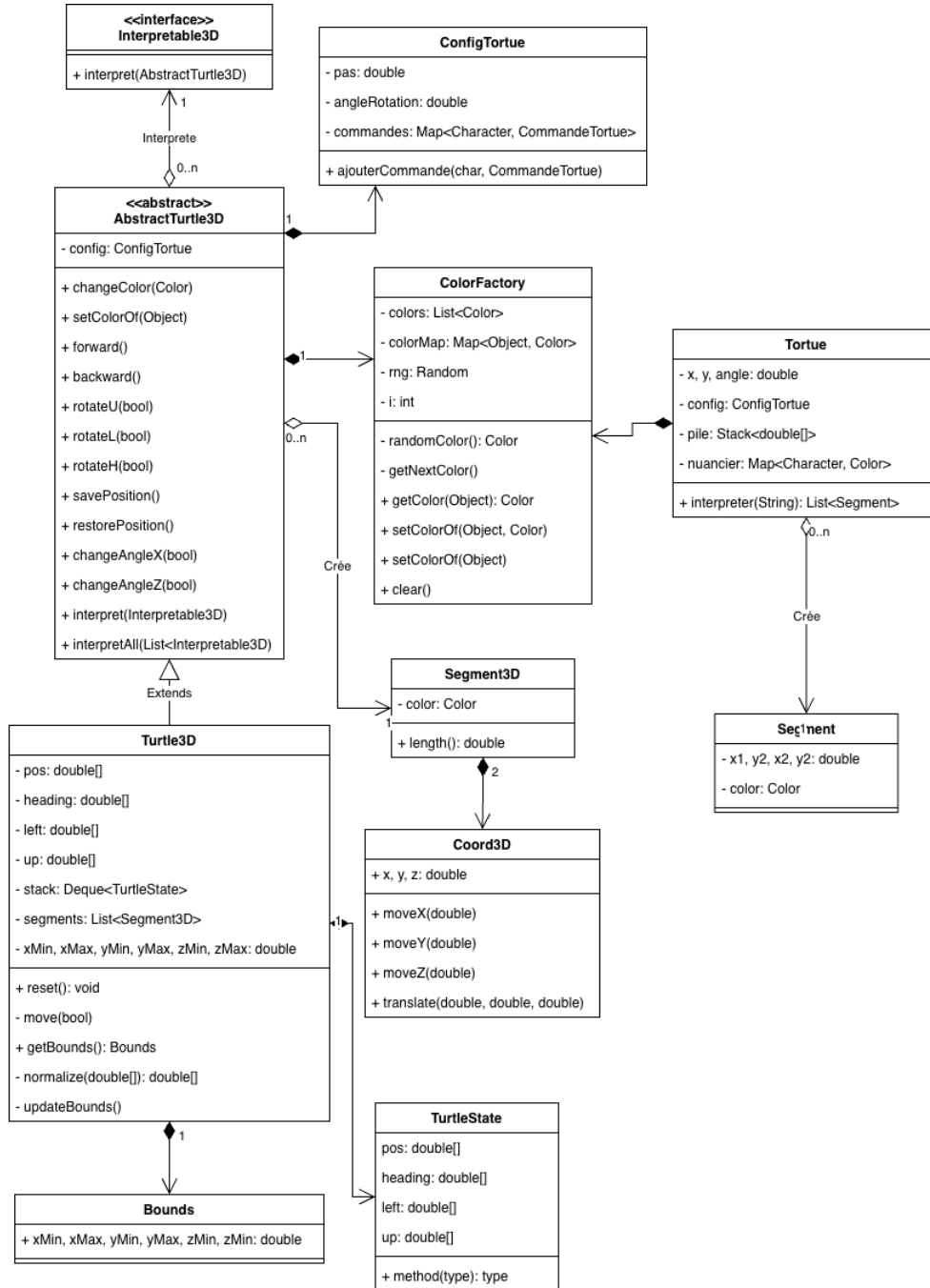


FIGURE 5 – Diagramme des classes du package turtle

tortue. Ces tortues produisent des listes de segments, qui sont traités dans les classes de vue pour effectuer l’affichage du système.

4.2.1 Tortue2D

La tortue 2-dimensionnelle a été créée en premier, et repose sur la valeur contenue dans un Symbole donné pour choisir le mouvement à effectuer. Cette architecture crée une dépendance forte entre les deux packages, et rend plus difficile la modification de la commande associée à un symbole particulier.

4.2.2 Tortue3D

Cette tortue a bénéficié des leçons apprises pendant le développement de la tortue 2D, et repose sur interface publique pour associer des commandes à des symboles. Ainsi, plutôt que de définir une commande directement dans la tortue, la tortue expose une API publique avec tous les mouvements qu’elle peut effectuer, que les classes devant déplacer la tortue peuvent appeler pour la déplacer. Enfin, l’association d’une commande à un symbole se fait par le biais d’une interface publique `Interpretable3D`, déléguant la responsabilité du choix de mouvement à la classe implémentatrice, ce qui limite l’interdépendance dans le paquetage.

4.3 Interface graphique

L’interface graphique est en majeure partie réalisée avec le module Swing de la bibliothèque standard de Java. Chaque composant fréquemment utilisé a été étendu afin de pouvoir fournir facilement une configuration par défaut de l’affichage. Un `Button`, par exemple, est un `JButton`, étendu avec des attributs statiques permettant ces changements.

4.3.1 Rendu 3D

Au fur et à mesure du développement du projet, il est vite devenu apparent que les fonctions de dessin de Swing, suffisantes pour un rendu 2-dimensionnel, n’étaient pas à la hauteur pour le rendu 3-dimensionnel. JavaFX a alors été choisi, et intégré au projet.

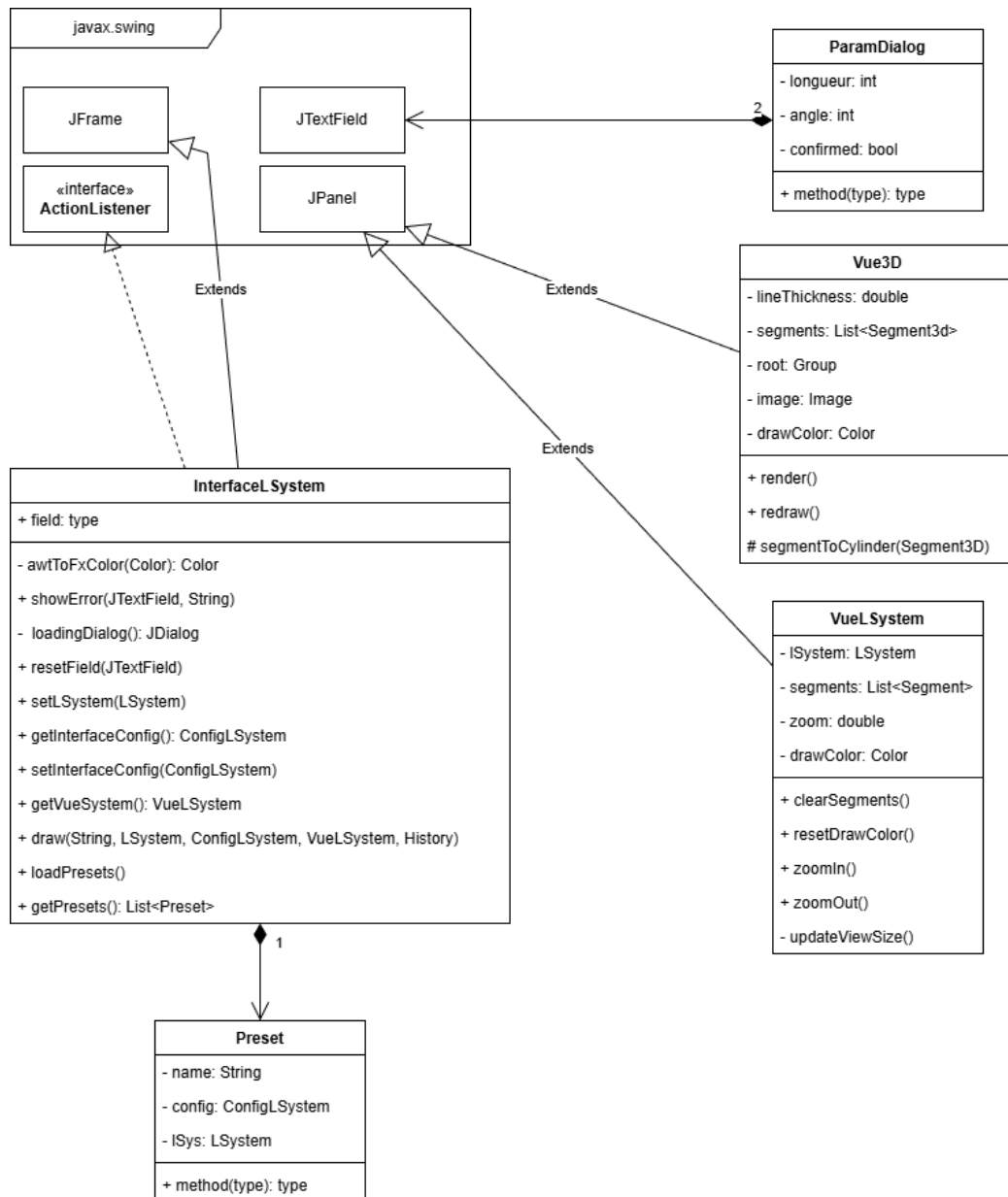


FIGURE 6 – Diagramme des classes du package ui

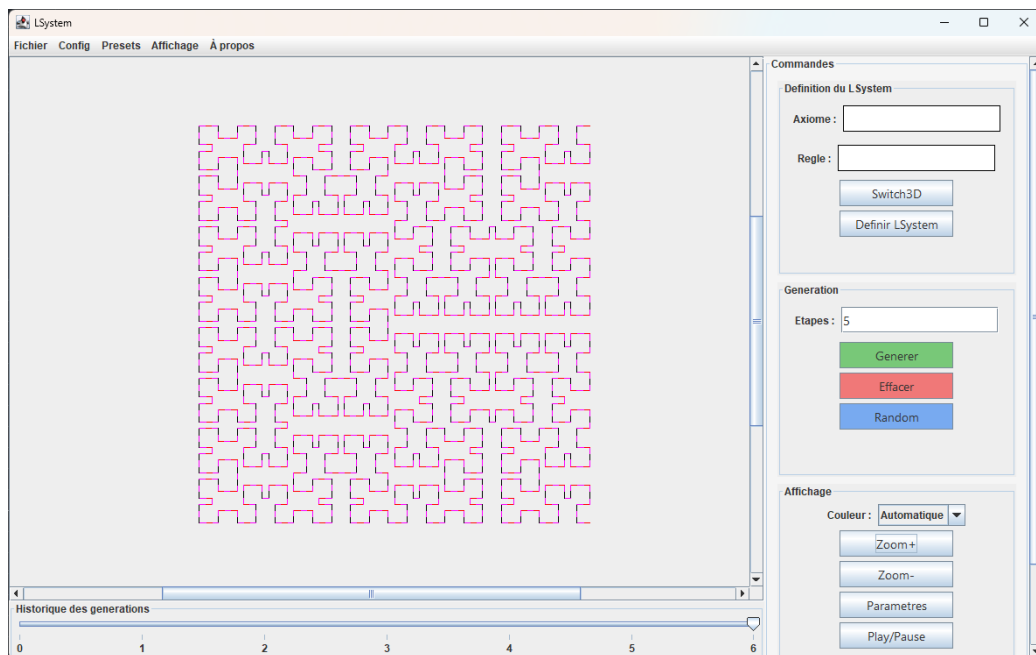


FIGURE 7 – Fenêtre principale de l’application, montrant une courbe de Hilbert

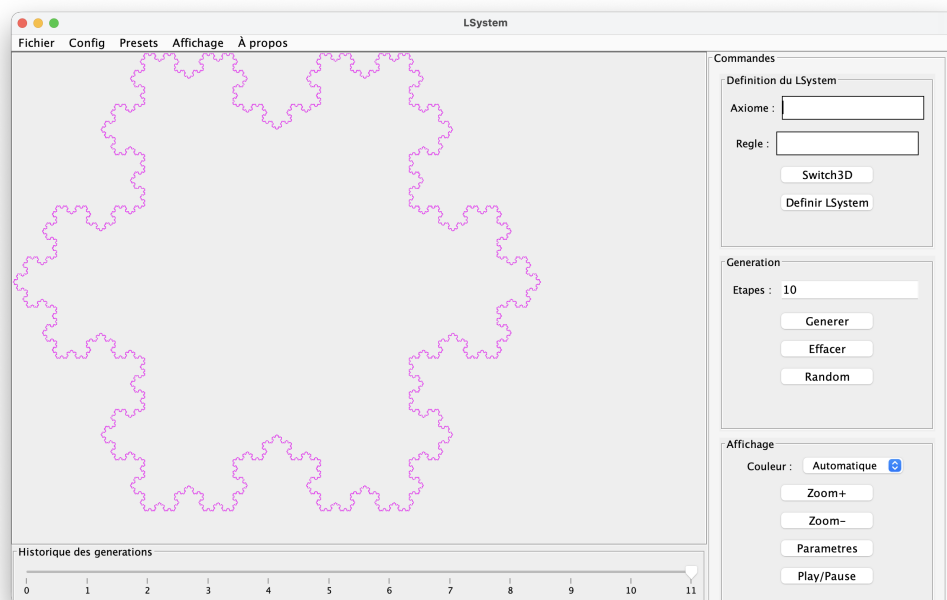


FIGURE 8 – Un flocon de Koch après 10 générations

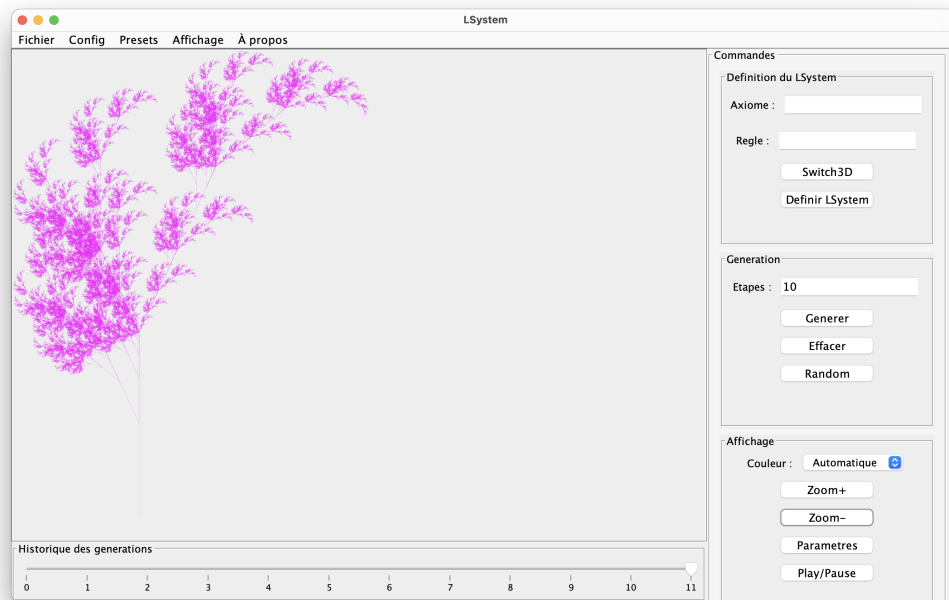


FIGURE 9 – Arbre simple après 10 générations

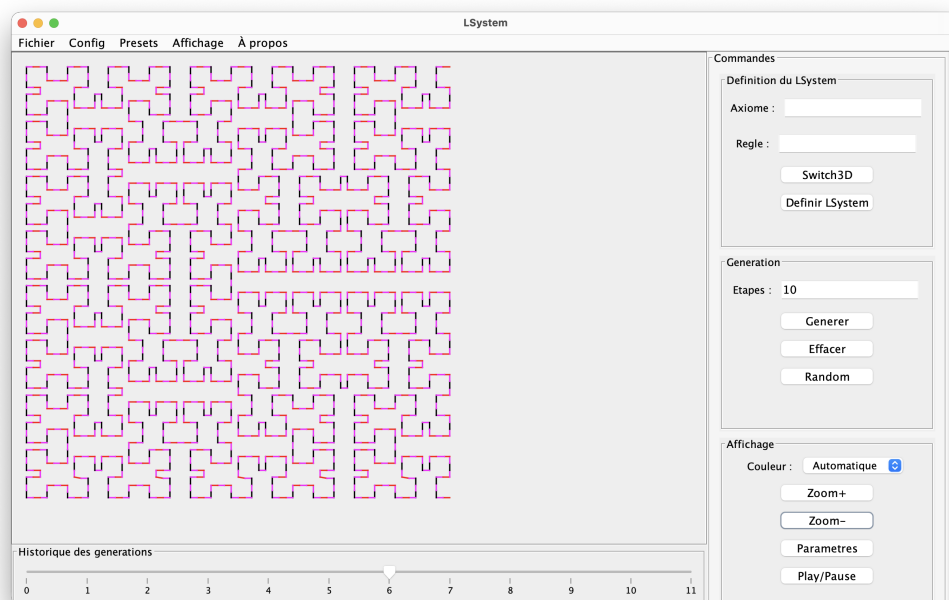


FIGURE 10 – Courbe de Hilbert après 6 générations

5 Expérimentations et usages

5.1 Performances

Système	Génération	Temps	Segments	Segments/s
Flocon de Koch 8	10	0.302411s	3145728	10e6/s
Arbre 9	10	0.42375s	1571328	3.7e6/s
Courbe de Hilbert 10	10	0.193248s	2097151	10.8e6/s

6 Conclusion

6.1 Récapitulatif des fonctionnalités principales

Nous avons réalisé les fonctionnalités suivantes :

- Un moteur de L-Système complet avec axiome, règles simples et contextuelles, génération itérative
- Une tortue graphique 2D avec interprétations de 6 symboles
- Une interface swing complète avec 10 L-Systèmes prédéfinis, historique, animation, zoom et couleurs.
- Une tortue 3D avec les trois matrices de rotation du cours
- Un affichage 3D en JavaFx avec des cylindres orientés dans l'espace
- Des tests unitaires avec les classes Symbol, RuleSet, LSystem, Coord3D, Segment3D.

6.2 Propositions d'améliorations